

Pointer in C

Syntax

- **C Pointers**

Pointers (pointer variables) are special variables that are used to store addresses rather than values.

- **Pointer Syntax**

Here is how we can declare pointers.

`int* p;` Here, we have declared a pointer p of int type.

You can also declare pointers in these ways.

`int *p1; int * p2;`

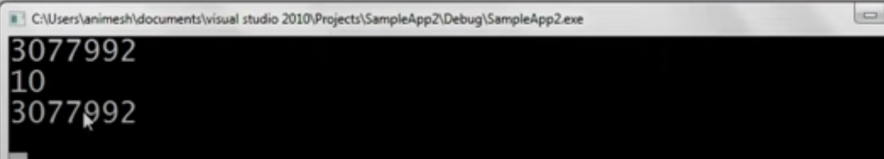
Let's take another example of declaring pointers.

`int* p1, p2;` Here, we have declared a pointer p1 and a normal variable p2.

Address and value

* is called the dereference operator (when working with pointers). It operates on a pointer and gives the value stored in that pointer.

```
#include<stdio.h>
int main()
{
    int a;
    int *p;
    a = 10;
    p = &a; // &a = address of a
    printf("%d\n",p);
    printf("%d\n",*p); // *p - value at address pointed by p
    printf("%d\n",&a);
}
```



The screenshot shows a Windows command prompt window with the title "C:\Users\animesh\documents\visual studio 2010\Projects\SampleApp2\Debug\SampleApp2.exe". The output of the program is displayed on three lines: 3077992, 10, and 3077992.

```
int b = 20;
*p = b; // Will the address in p change to point b
printf("Address of P is %d\n",p);
printf("Value at p is %d\n",*p);
}
```

Example

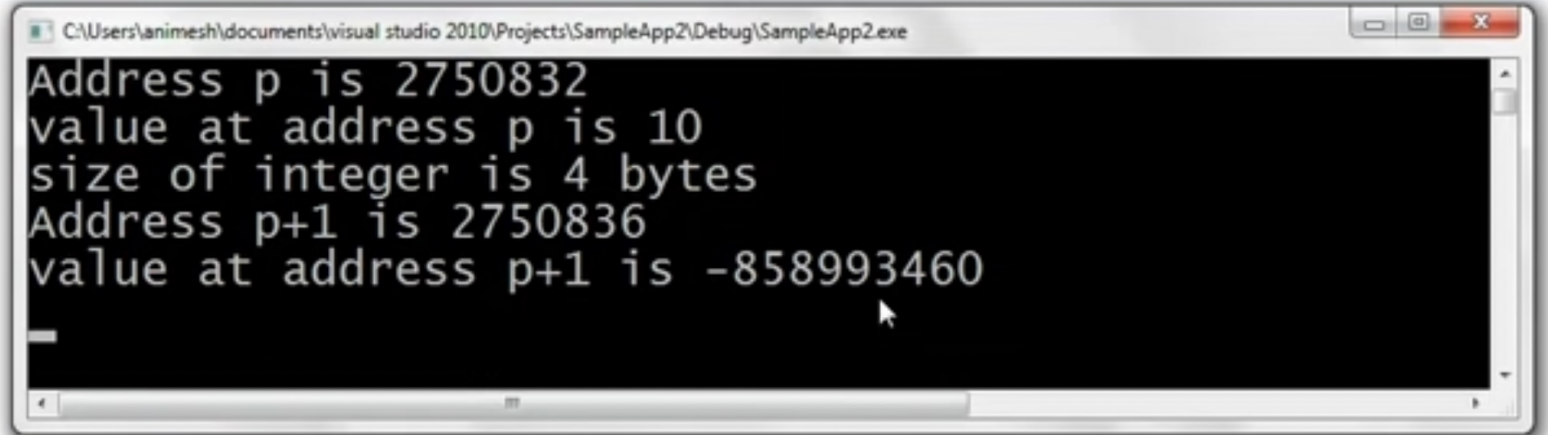
```
#include <stdio.h>
int main(void) {
    int a=10;
    int *p=&a;
    printf("%d \n",p)
    ;printf("%d \n",*p)
    ;printf("%d \n",a);
    int b=20;
    *p=b;      What if we write p=&b;
    printf("%d \n",p);
    printf("%d \n",*p);
    printf("%d \n",a);
    printf("%d \n",b);
    printf("%d \n",&b);
    printf("%d \n",&a);    return 0;}
```

Output:

```
299915360
10
10
299915360
20 20 20
299915364
299915360
```

Pointer Arithmetic

```
#include<stdio.h>
int main()
{
    int a = 10;
    int *p;
    p = &a;
    // Pointer arithmetic
    printf("Address p is %d\n",p);
    printf("value at address p is %d\n",*p);
    printf("size of integer is %d bytes\n",sizeof(int));
    printf("Address p+1 is %d\n",p+1);
    printf("value at address p+1 is %d\n",*(p+1));
}
```

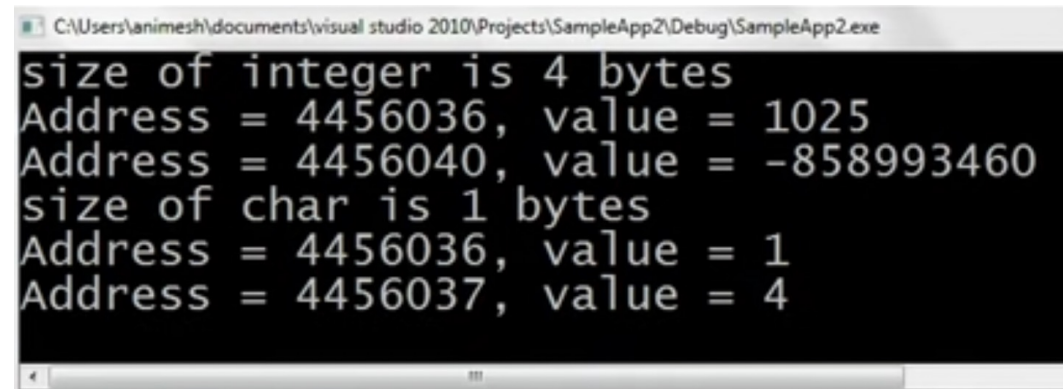


A screenshot of a Windows command prompt window titled "C:\Users\animesh\documents\visual studio 2010\Projects\SampleApp2\Debug\SampleApp2.exe". The window displays the output of the C program, showing the memory address of variable p, the value stored at that address, the size of an integer, and the address and value at the next memory location (p+1).

```
Address p is 2750832
value at address p is 10
size of integer is 4 bytes
Address p+1 is 2750836
value at address p+1 is -858993460
```

Pointer Types

```
#include<stdio.h>
int main()
{
    int a = 1025;
    int *p;
    p = &a;
    printf("size of integer is %d bytes\n",sizeof(int));
    printf("Address = %d, value = %d\n",p,*p);
    printf("Address = %d, value = %d\n",p+1,*p+1));
    char *p0;
    p0 = (char*)p; // typecasting
    printf("size of char is %d bytes\n",sizeof(char));
    printf("Address = %d, value = %d\n",p0,*p0);
    printf("Address = %d, value = %d\n",p0+1,*p0+1));
    // 1025 = 00000000 00000000 00000100 00000001
}
```

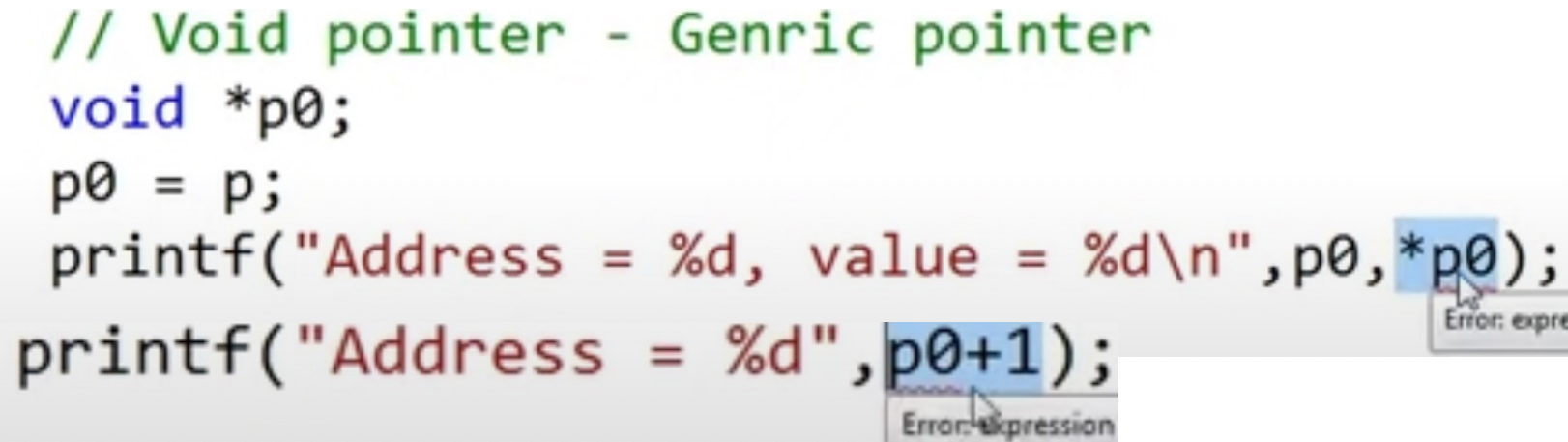


```
C:\Users\animesh\documents\visual studio 2010\Projects\SampleApp2\Debug\SampleApp2.exe
size of integer is 4 bytes
Address = 4456036, value = 1025
Address = 4456040, value = -858993460
size of char is 1 bytes
Address = 4456036, value = 1
Address = 4456037, value = 4
```

Void Pointer

- The void pointer in C is a **pointer that is not associated with any data types**. It points to some data location in the storage. This means that it points to the address of variables. It is also called the general purpose pointer.

```
// Void pointer - Genric pointer
void *p0;
p0 = p;
printf("Address = %d, value = %d\n", p0, *p0);
printf("Address = %d", p0+1);
```

A screenshot of a code editor showing C code. The code defines a void pointer p0, assigns it a value p, and uses it in two printf statements. The first printf statement has a blue highlight on *p0 and a tooltip that says "Error: expression". The second printf statement has a blue highlight on p0+1 and a tooltip that says "Error: expression".

Error: expression

Error: expression

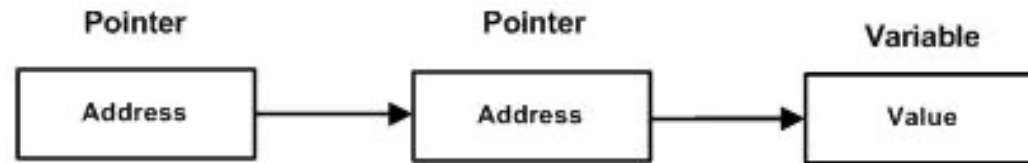
Pointer to pointer

A pointer to a pointer is a form of multiple indirection, or a chain of pointers. Normally, a pointer contains the address of a variable. When we define a pointer to a pointer, the first pointer contains the address of the second pointer, which points to the location that contains the actual value as shown below.

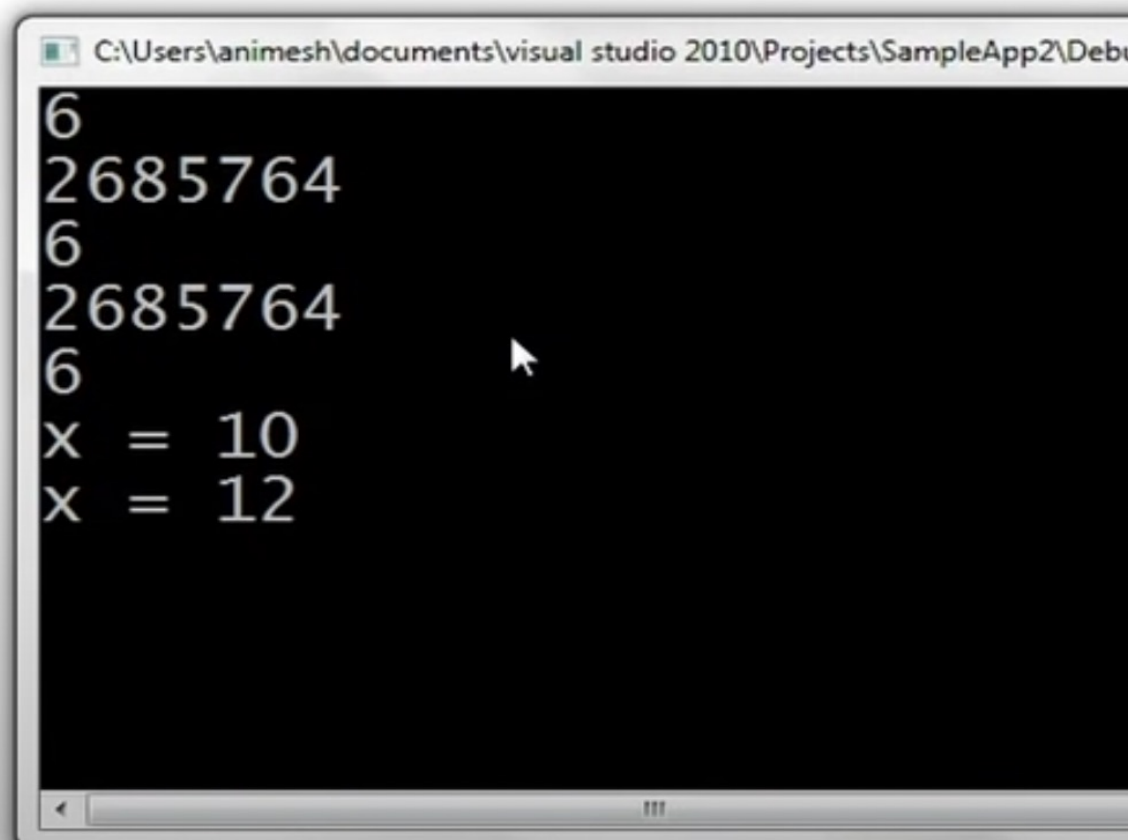
A variable that is a pointer to a pointer must be declared as such. This is done by placing an additional asterisk in front of its name. For example, the following declaration declares a pointer to a pointer of type int –

```
int **var;
```

When a target value is indirectly pointed to by a pointer to a pointer, accessing that value requires that the asterisk operator be applied twice



```
// pointers to pointers
#include<stdio.h>
int main()
{
    int x = 5;
    int* p = &x;
    *p = 6;
    int** q = &p;
    int*** r = &q;
    printf("%d\n", *p);
    printf("%d\n", *q);
    printf("%d\n", **q);
    printf("%d\n", **r);
    printf("%d\n", ***r);
    ***r = 10;
    printf("x = %d\n", x);
    **q = *p + 2;
    printf("x = %d\n", x);
}
```



The screenshot shows a Windows command prompt window with the following output:

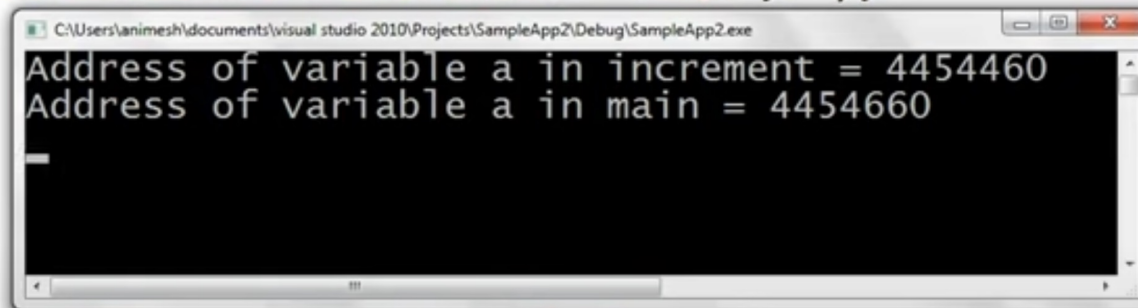
```
C:\Users\animesh\documents\visual studio 2010\Projects\SampleApp2\Debu
6
2685764
6
2685764
6
x = 10
x = 12
```

The output corresponds to the printf statements in the code: the first three lines print the values of *p, *q, and **q respectively, all of which are 6. The next line prints the value of **r, which is 2685764. The following line prints the value of ***r, which is also 2685764. The next line prints the value of x, which is 10. The final line prints the value of x, which is 12.

Pointer as Function arguments- Call by Reference

For a function to effect call-by-reference, **pointers must be used in the parameter list in the function definition**. Then, when the function is called, addresses of variables must be passed as arguments.

```
#include<stdio.h>
void Increment(int a)
{
    a = a+1;
    printf("Address of variable a in increment = %d\n",&a);
}
int main()
{
    int a;
    a = 10;
    Increment(a);
    printf("Address of variable a in main = %d\n",&a);
    //printf("a = %d",a);
}
```



Call by value

```
#include<stdio.h>
void Increment(int *p)
{
    *p = (*p) + 1;
}
int main()
{
    int a;
    a = 10;
    Increment(&a);
    printf("a = %d",a);
}
```

Call by reference

Pointers and Arrays

```
// Pointers and Arrays
#include<stdio.h>
int main()
{
    int A[] = {2,4,5,8,1};
    int i;
    int *p = A;
    for(int i = 0; i<5; i++)
    {
        printf("Address = %d\n", &A[i]);
        printf("Address = %d\n", A+i);
        printf("value = %d\n", A[i]);
        printf("value = %d\n", *(A+i));
    }
}
```

Example

```
// Arrays as function arguments
#include<stdio.h>
int SumOfElements(int A[], int size)
{
    int i, sum = 0;
    for(i = 0; i < size; i++)
    {
        sum += A[i];
    }
    return sum;
}
int main()
{
    int A[] = {1, 2, 3, 4, 5};
    int size = sizeof(A) / sizeof(A[0]);
    int total = SumOfElements(A, size);
    printf("Sum of elements = %d\n", total);
}
```

Sum of elements = 15

```
// Arrays as function arguments
#include<stdio.h>
int SumOfElements(int A[])
{
    int i, sum = 0;
    int size = sizeof(A) / sizeof(A[0]);
    for(i = 0; i < size; i++)
    {
        sum += A[i];
    }
    return sum;
}
int main()
{
    int A[] = {1, 2, 3, 4, 5};
    int total = SumOfElements(A);
    printf("Sum of elements = %d\n", total);
}
```

Sum of elements = 1

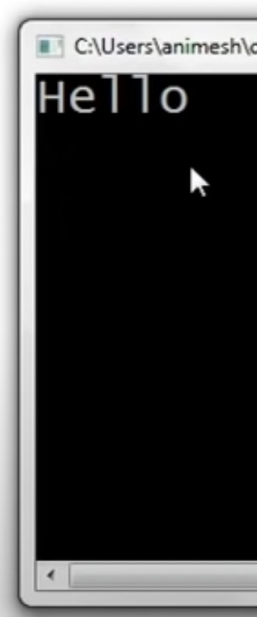
Character Arrays and Pointers

```
//character arrays and pointers
#include<stdio.h>
void print(char* C)
{
    int i = 0;
    while(C[i] != '\0')
    {
        printf("%c",C[i]);
        i++;
    }
    printf("\n");
}
int main()
{
    char C[20] = "Hello";
    print(C);
}
```

Using array elements

```
//character arrays and pointers
#include<stdio.h>
void print(char* C)
{
    while(*C != '\0')
    {
        printf("%c",*C);
        C++;
    }
    printf("\n");
}
int main()
{
    char C[20] = "Hello";
    print(C);
}
```

Using pointer variable



Changing value of a character Array

```
//character arrays and pointers
#include<stdio.h>
#include<string.h>
void print(char *C)
{
    while(*C != '\0')
    {
        printf("%c",*C);
        C++;
    }
    printf("\n");
}
int main()
{
    //char C[20] = "Hello"; // string gets stored in the space for array
    char *C = "Hello"; // string gets stored as compile time constant
    C[0] = 'A';
    printf("Hello World");
    print(C);
}
```

Value cannot be changed

```
//character arrays and pointers
#include<stdio.h>
#include<string.h>
void print(char *C)
{
    C[0] = 'A';
    while(*C != '\0')
    {
        printf("%c",*C);
        C++;
    }
    printf("\n");
}
int main()
{
    char C[20] = "Hello";
    print(C);
}
```

Can be changed

Pointer and 2D arrays

```
int B[2][3]
```

$\left. \begin{array}{l} B[0] \\ B[1] \end{array} \right\} \rightarrow \text{1-D arrays of 3 integers}$

```
int (*p)[3] = B;
```

```
Print B or &B[0] // 400
```

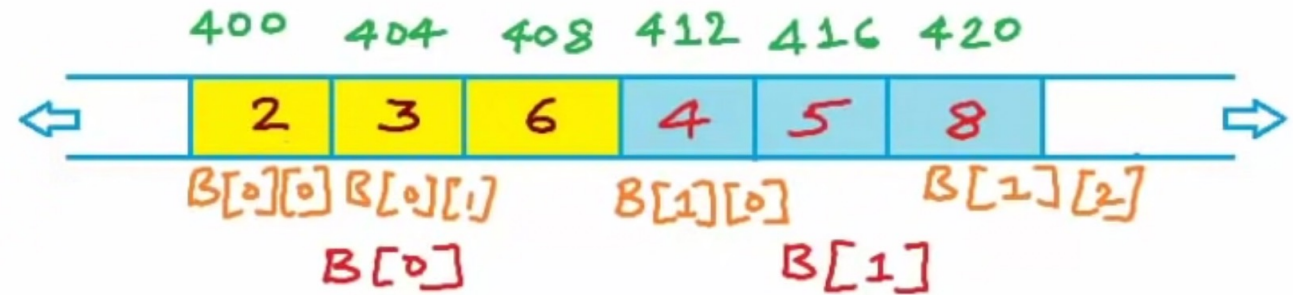
```
Print *B or B[0] or &B[0][0] // 400
```

```
Print B+1 or &B[1] // 412
```

```
Print *(B+1) or B[1] or &B[1][0] // 412
```

```
Print *(B+1)+2 or B[1]+2 or &B[1][2] // 420
```

```
Print *(*B+1) // 3
```



$$\begin{aligned} B[i][j] &= *(B[i] + j) \\ &= *(*B + i) + j \end{aligned}$$

Example

```
// Pointers and multi-dimensional arrays
#include<stdio.h>
void Func(int (*A)[2][2]) // Argument: 3-D array of integers
{
}
int main()
{
    int C[3][2][2]={{{2,5},{7,9}},
                    {{3,4},{6,1}},
                    {{0,8},{11,13}}};

    int A[2] = {1,2};
    int B[2][3] ={{2,4,6},{5,7,8}}; // B returns int (*)(3)
    Func(C);
}
```

Dynamically allocated Array

The “**malloc**” or “**memory allocation**” method in C is used to dynamically allocate a single large block of memory with the specified size. It returns a pointer of type void which can be cast into a pointer of any form. It doesn't Initialize memory at execution time so that it has initialized each block with the default garbage value initially.

Syntax:

ptr = (cast-type*) malloc(byte-size)

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    int n;
    printf("Enter size of array\n");
    scanf("%d",&n);
    int *A = (int*)malloc(n*sizeof(int)); //dynamically allocated array
    for(int i = 0;i<n;i++)
    {
        A[i] = i+1;
    }
    for(int i =0;i<n;i++)
    {
        printf("%d ",A[i]);
    }
}
```

Pointer as function returns

```
// Pointers as function returns
#include<stdio.h>
#include<stdlib.h>
int Add(int* a,int* b){ // Called function
    int c = (*a) + (*b);
    return c;
}
int main() { //Calling function
    int a = 2, b =4;
    printf("Address of a in main = %d\n",&a);
    int c = Add(&a,&b); // a and b are integers local to Main
    printf("Sum = %d\n",c);
}
```

Output: Sum = 6

```
// Pointers as function returns
#include<stdio.h>
#include<stdlib.h>
int *Add(int* a,int* b){ // Called funtion - returns pointer tp integer
    int c = (*a) + (*b);
    return &c;
}
int main() { //Calling function
    int a = 2, b =4;
    printf("Address of a in main = %d\n",&a);
    int* ptr = Add(&a,&b); // a and b are integers local to Main
    printf("Sum = %d\n",*ptr);
}
```

Output: Sum = 6

Solution: `int* c = (int*)malloc(sizeof(int));`

```
// Pointers as function returns
#include<stdio.h>
#include<stdlib.h>
void PrintHelloWorld(){
    printf("Hello World\n");
}
int *Add(int* a,int* b){ // Called function - returns pointer to integer
    int c = (*a) + (*b);
    return &c;
}
int main() { //Calling function
    int a = 2, b =4;
    int* ptr = Add(&a,&b); // a and b are integers local to Main
    PrintHelloWorld();
    printf("Sum = %d\n",*ptr);
}
```

```
Hello World
Sum = -858993460
```